

PRACTICAL RECORD

Python Flask Web Application

Name: S Svava Haasini

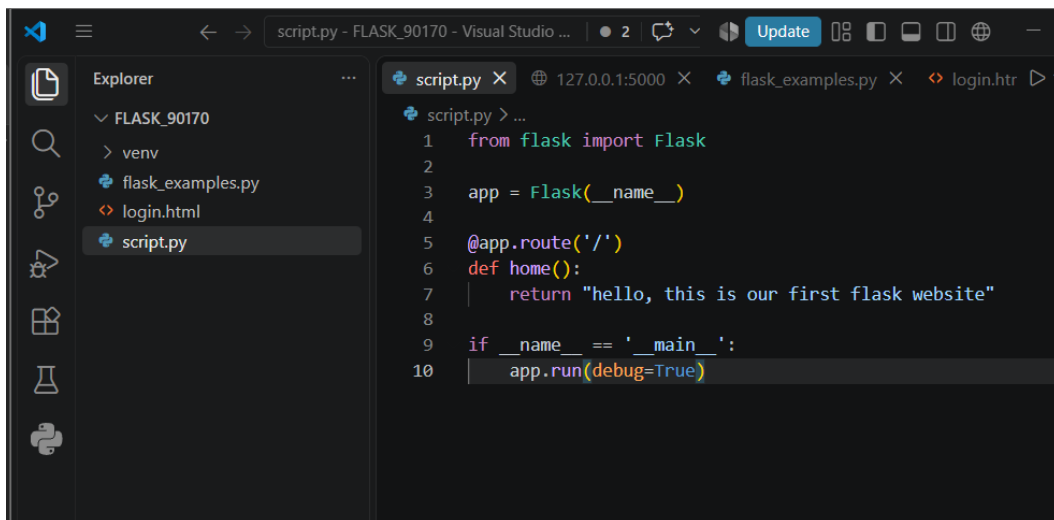
Roll Number: 2520090170

Aim:

To develop Flask web applications demonstrating basic routing, variable URL routing, dynamic URL building with `url_for()`, and GET and POST methods for form submission.

1. First Flask Application

A simple Flask application is created with a basic route to display a welcome message.

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a project named 'FLASK_90170' with a 'venv' folder and files 'flask_examples.py', 'login.html', and 'script.py'. The main editor window displays the code for 'script.py'. The code imports Flask, creates an app, defines a home route, and runs the app with debug mode enabled. The file explorer on the right shows 'script.py', 'flask_examples.py', and 'login.html'.

```
1  from flask import Flask
2
3  app = Flask(__name__)
4
5  @app.route('/')
6  def home():
7      return "hello, this is our first flask website"
8
9  if __name__ == '__main__':
10     app.run(debug=True)
```

Figure 1: script.py - First Flask Application Code

The Flask app runs on localhost:5000 and displays "hello, this is our first flask website".

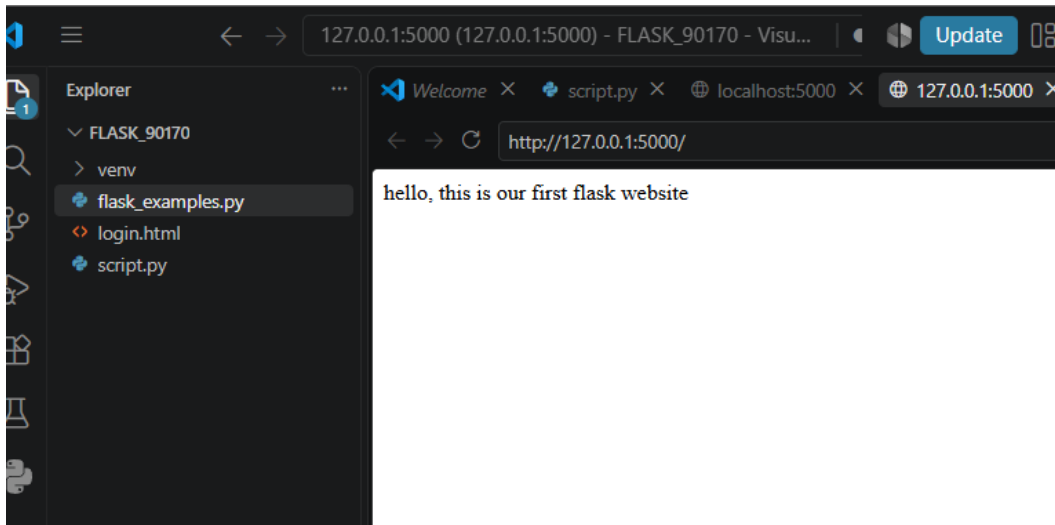
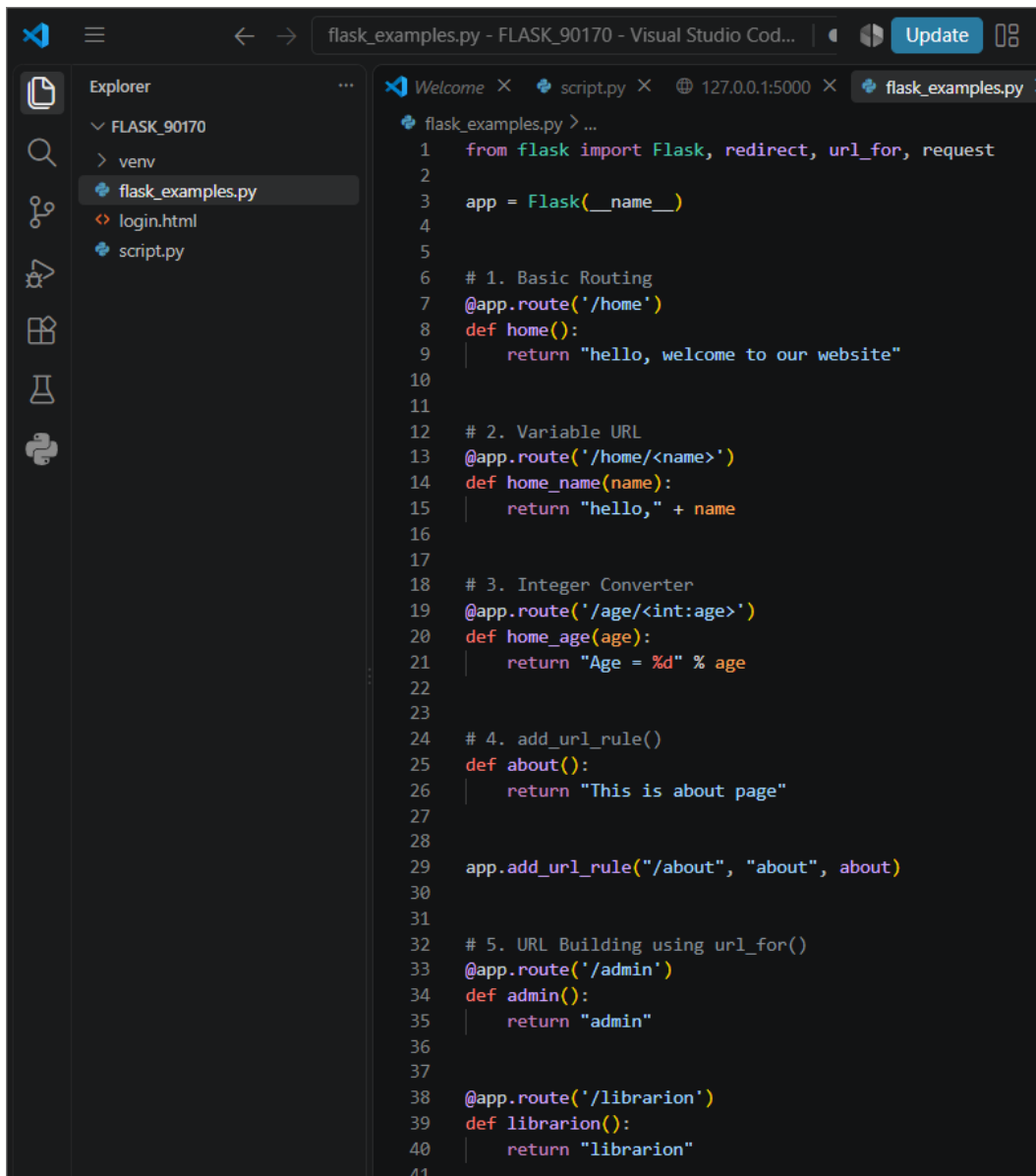


Figure 2: First Flask Application Output

2. Basic Flask Routing

Basic routing is used to map URLs to functions that return responses.

The image shows a screenshot of the Visual Studio Code editor. The Explorer sidebar on the left shows a project named 'FLASK_90170' with a 'venv' folder and files 'flask_examples.py', 'login.html', and 'script.py'. The main editor window displays the 'flask_examples.py' file. The code defines a Flask application with several routes: a basic route for '/home' that returns 'hello, welcome to our website', a route for '/home/<name>' that returns 'hello,' followed by the name, a route for '/age/<int:age>' that returns 'Age = <age>', a route for '/about' that returns 'This is about page', and two more routes for '/admin' and '/librarian' that return 'admin' and 'librarian' respectively. The code is numbered from 1 to 41.

```
1 from flask import Flask, redirect, url_for, request
2
3 app = Flask(__name__)
4
5
6 # 1. Basic Routing
7 @app.route('/home')
8 def home():
9     return "hello, welcome to our website"
10
11
12 # 2. Variable URL
13 @app.route('/home/<name>')
14 def home_name(name):
15     return "hello," + name
16
17
18 # 3. Integer Converter
19 @app.route('/age/<int:age>')
20 def home_age(age):
21     return "Age = %d" % age
22
23
24 # 4. add_url_rule()
25 def about():
26     return "This is about page"
27
28
29 app.add_url_rule("/about", "about", about)
30
31
32 # 5. URL Building using url_for()
33 @app.route('/admin')
34 def admin():
35     return "admin"
36
37
38 @app.route('/librarian')
39 def librarian():
40     return "librarian"
41
```

Figure 3: flask_examples.py - Basic Routing (/home route)

When accessing /home, the application returns "hello, welcome to our website".

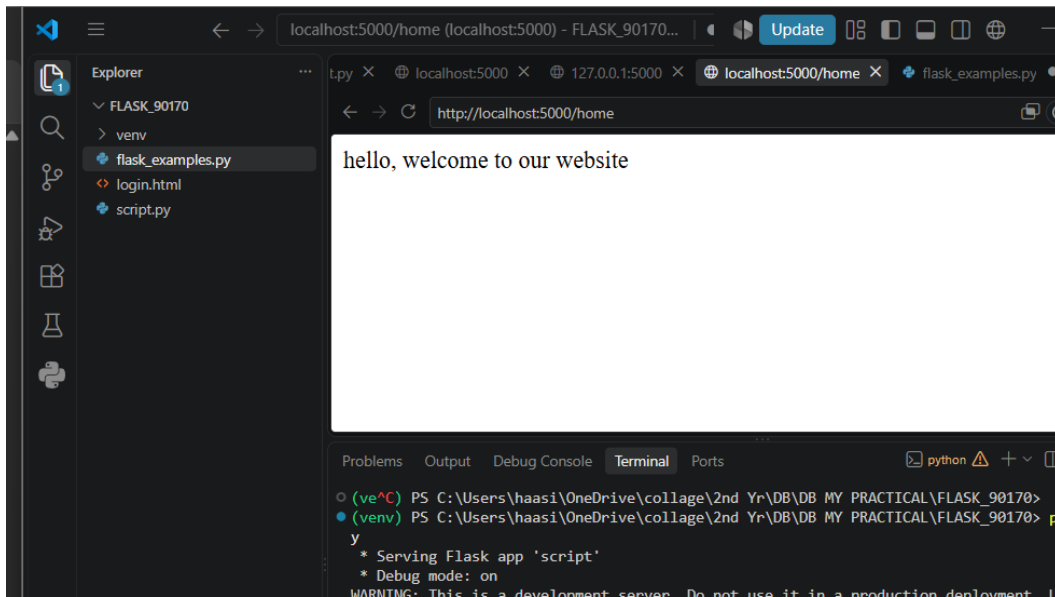
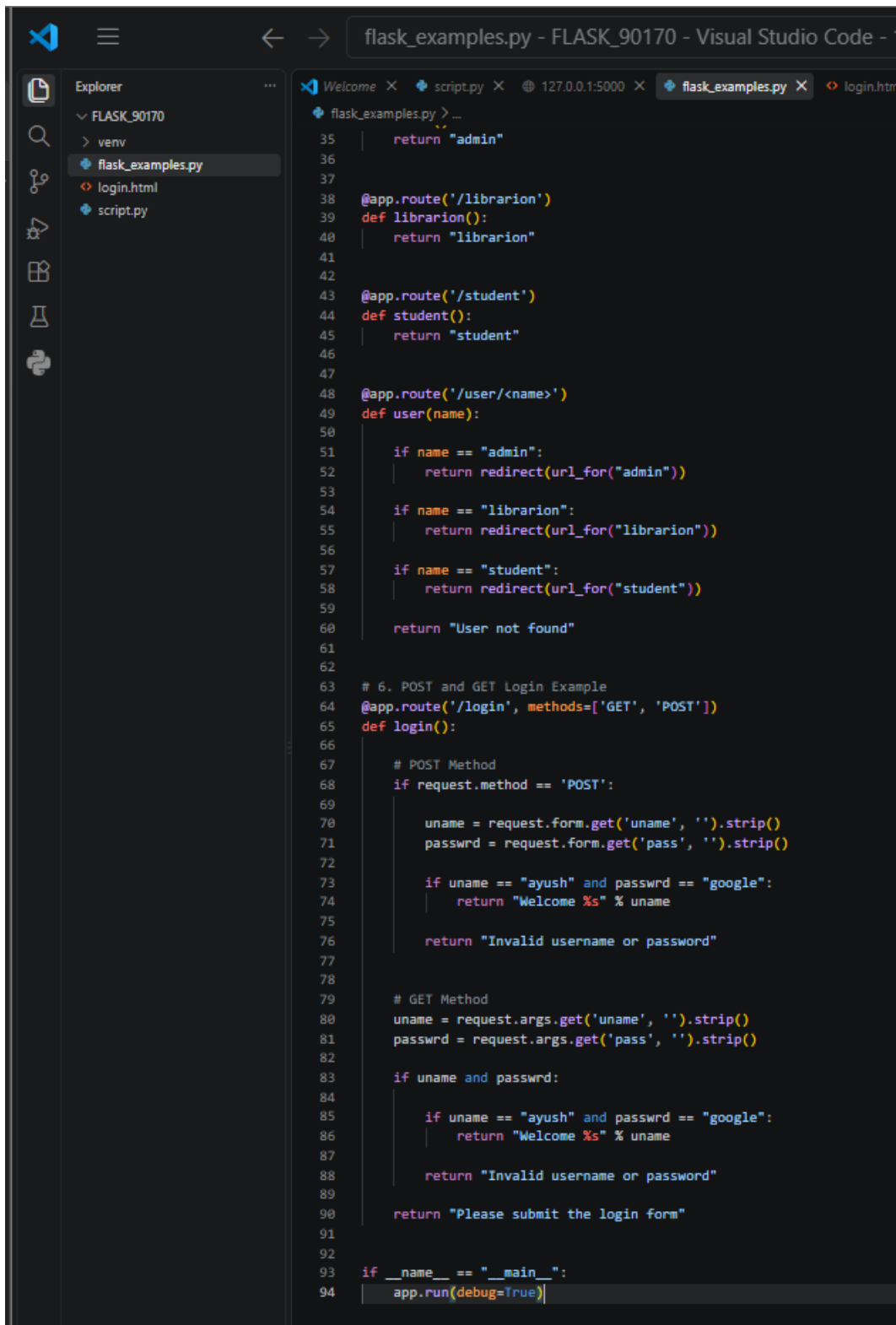


Figure 4: Basic Routing Output

3. Variable URL Routing

Variable URL routing allows dynamic URLs with parameters extracted from the URL path.



The image shows a Visual Studio Code editor window titled "flask_examples.py - FLASK_90170 - Visual Studio Code". The Explorer sidebar on the left shows a project named "FLASK_90170" with files "venv", "flask_examples.py", "login.html", and "script.py". The main editor displays the code for "flask_examples.py".

```
35 |     return "admin"
36 |
37 |
38 | @app.route('/librarian')
39 | def librarian():
40 |     return "librarian"
41 |
42 |
43 | @app.route('/student')
44 | def student():
45 |     return "student"
46 |
47 |
48 | @app.route('/user/<name>')
49 | def user(name):
50 |
51 |     if name == "admin":
52 |         return redirect(url_for("admin"))
53 |
54 |     if name == "librarian":
55 |         return redirect(url_for("librarian"))
56 |
57 |     if name == "student":
58 |         return redirect(url_for("student"))
59 |
60 |     return "User not found"
61 |
62 |
63 | # 6. POST and GET Login Example
64 | @app.route('/login', methods=['GET', 'POST'])
65 | def login():
66 |
67 |     # POST Method
68 |     if request.method == 'POST':
69 |
70 |         uname = request.form.get('uname', '').strip()
71 |         passwd = request.form.get('pass', '').strip()
72 |
73 |         if uname == "ayush" and passwd == "google":
74 |             return "Welcome %s" % uname
75 |
76 |         return "Invalid username or password"
77 |
78 |
79 |     # GET Method
80 |     uname = request.args.get('uname', '').strip()
81 |     passwd = request.args.get('pass', '').strip()
82 |
83 |     if uname and passwd:
84 |
85 |         if uname == "ayush" and passwd == "google":
86 |             return "Welcome %s" % uname
87 |
88 |         return "Invalid username or password"
89 |
90 |     return "Please submit the login form"
91 |
92 |
93 | if __name__ == "__main__":
94 |     app.run(debug=True)
```

Figure 5: flask_examples.py - Variable URL Routing (/home/<name> route)

Accessing `/home/Haasini` returns "hello,Haasini" by extracting the name parameter from the URL.

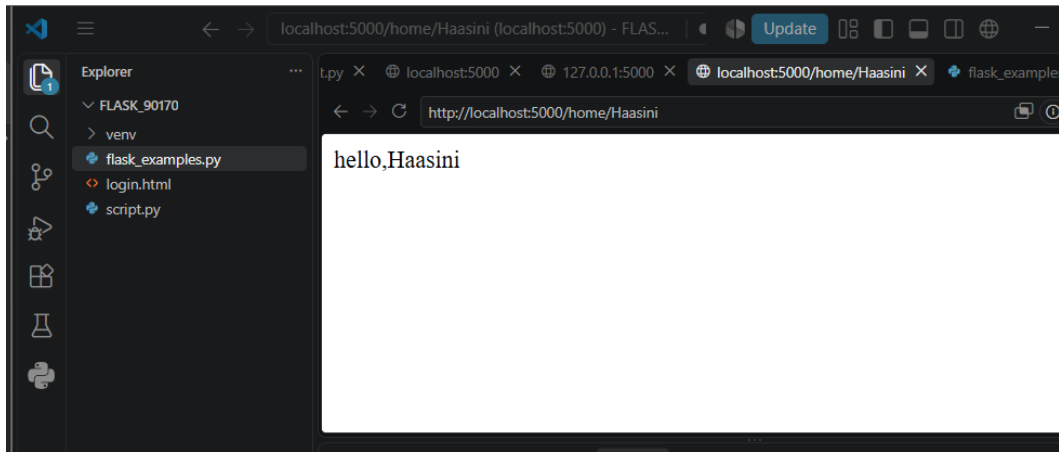


Figure 6: Variable URL Routing Output

4. Dynamic URL Building using `url_for()`

The `url_for()` function is used to dynamically build URLs and perform redirects to different routes.

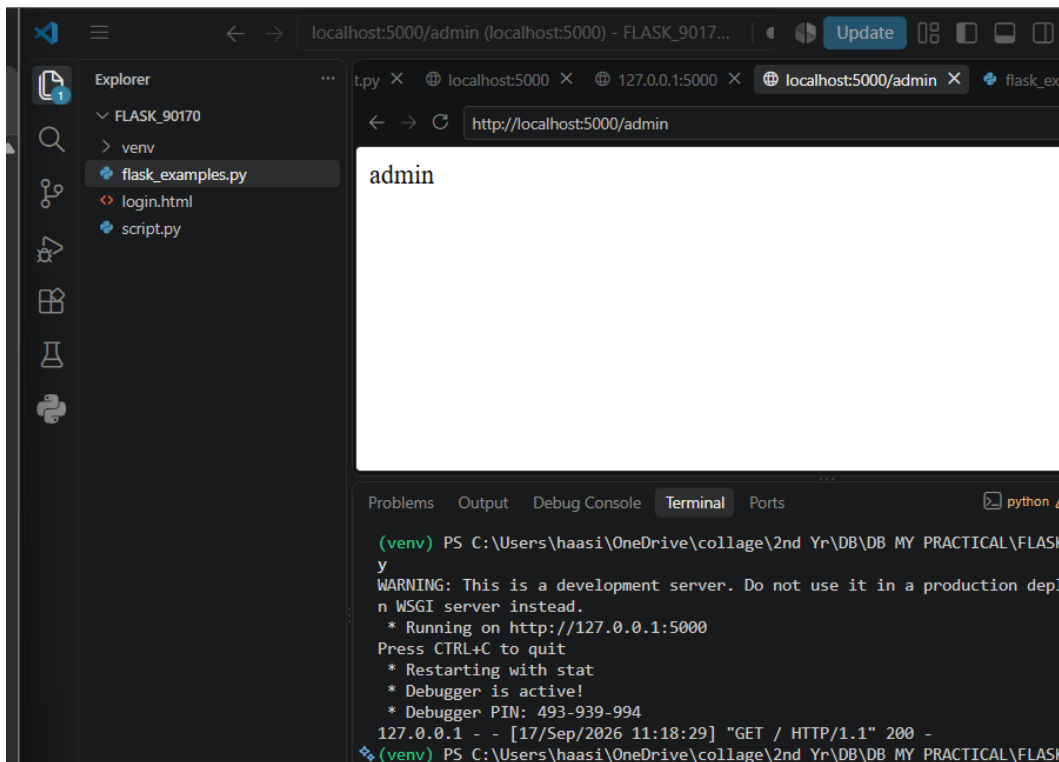
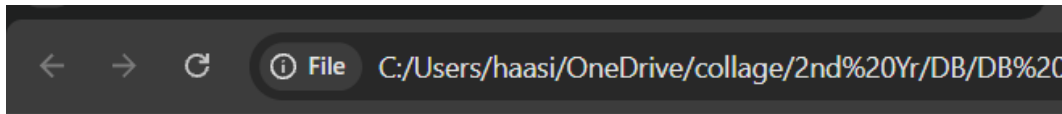


Figure 7: Dynamic URL Building Output (`/user/admin` route)

5. Flask Login Form

An HTML login form is created with POST and GET methods for form submission and credential validation.



POST Method

Name	ayush
Password	
Submit POST	

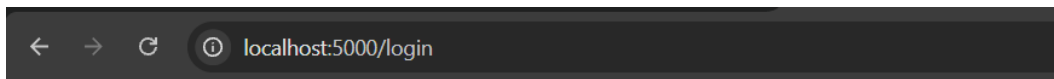
GET Method

Name	
Password	
Submit GET	

Figure 8: login.html - Login Form with POST and GET Methods

6. POST Method

The POST method submits form data securely to the server without exposing credentials in the URL.



Welcome ayush

Figure 9: POST Method - Login Success Response

7. GET Method

The GET method passes form data through the URL query string as parameters.



POST Method

Name	ayush
Password	
Submit POST	

GET Method

Name	ayush
Password	
Submit GET	

Figure 10: GET Method - Form with Query Parameters

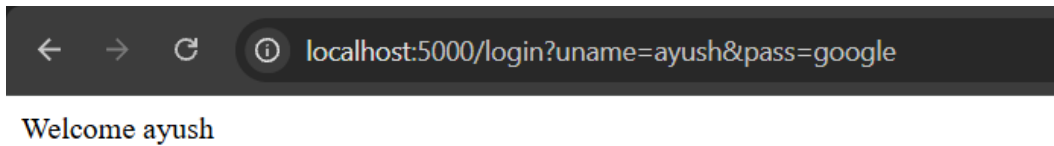


Figure 11: GET Method - Login Success with URL Query String

Result:

The Flask programs were executed successfully, demonstrating basic routing, variable routing, dynamic URL building, and GET and POST methods.